

01 — Program Structure

The minimum

```
#include <iostream>

int main() {
    std::cout << "hello\n";
    return 0;
}
```

Four things differ from Python and none are cosmetic.

`main` is mandatory and returns an `int`

Python runs a file top to bottom. C++ does not: the operating system hands control to exactly one function, `main`, and everything else happens because `main` reached it.

The returned `int` is the process exit code — `0` means success, anything else means failure. Shell scripts and CI systems read it. `return 0;` at the end of `main` is optional (the compiler inserts it), but writing it is a habit worth having.

`#include` is not `import`

```
#include <iostream>
```

`import` in Python loads a module object at run time. `#include` is handled by the **preprocessor** before compilation even starts: it takes the named file and pastes its entire contents into your source, textually.

That is genuinely all it does. The consequences matter, and Module 07 covers them:

- Including the same header twice pastes it twice, which is why headers need include guards.
- More includes means more text to compile, which is why C++ builds are slow.
- `<>` searches the system include paths; `"` searches relative to the current file first.

Statements end with semicolons; blocks use braces

Python uses indentation for structure. C++ uses `{ }` and ignores whitespace entirely. Indentation in C++ is a message to humans and carries no meaning for the compiler.

Missing a `;` produces an error message that often points at the *next* line, because that is where the compiler noticed the problem. This confuses everyone at first; expect it.

`std::cout` and namespaces

```
std::cout << "value: " << x << "\n";
```

`std` is the standard library namespace, `::` is the scope operator, and `<<` chains outputs left to right.

You will see `using namespace std;` in tutorials, which lets you write `cout` instead of `std::cout`. Avoid it. It pulls every standard library name into scope and causes collisions that produce spectacular error messages. Typing `std::` is cheap.

Prefer `"\n"` over `std::endl`. `endl` also flushes the output buffer, which is slower and almost never what you want.

Comments

```
// single line

/* multi
   line */
```

Why This Matters for CUDA

A `.cu` file is a C++ file with extra keywords. It has a `main` that runs on the CPU, includes that get textually pasted, and functions marked `__global__` that run on the GPU. The structure above is exactly the structure you will be reading — the only addition is that some functions execute on a different processor.